Cahier des Charges - API Bancaire

Étudiant : NGA YENDE ALI JUNIOR
Niveau : L3
Filière : ICT4D
Matière : ICT304 - Software Testing and Quality Assurance

1. Contexte et objectifs

Conception d'une API REST pour un systeme de transactions bancaires permettant aux clients de gerer leurs comptes, effectuer des depots et des retraits.

2. Besoins

2.1 Besoins fonctionnels

ID	Besoin	Priorite
BF01	Creer un compte bancaire	Haute
BF02	Consulter la liste des comptes	Haute
BF03	Consulter un compte par ID	Haute
BF04	Effectuer un depot sur un compte	Haute
BF05	Effectuer un retrait sur un compte	Haute
BF06	Consulter l'historique des transactions	Moyenne

2.2 Besoins non fonctionnels

ID	Besoin	Priorite
BNF01	Temps de reponse < 500ms par requete	Haute
BNF02	Validation des donnees en entree	Haute
BNF03	Gestion des erreurs avec codes HTTP appropries	Haute
BNF04	Un retrait ne peut pas rendre le solde negatif	Haute
BNF05	API documentee (endpoints clairs et coherents)	Moyenne
BNF06	Architecture extensible (ajout futur d'auth, BDD)	Moyenne

3. Acteurs

Acteur	Description
Client	Utilisateur qui cree un compte, depose et retire de l'argent
Systeme (API)	Traite les requetes, valide les donnees, gere les soldes
Administrateur	(Futur) Supervise les comptes et transactions

---

4. Cas d'utilisation

CU01 - Creer un compte

- **Acteur** : Client
- **Pre-condition** : Aucune
- **Scenario principal** :
    1. Le client envoie une requete POST /api/comptes avec son nom et prenom
    2. Le systeme valide les donnees
    3. Le systeme cree le compte avec un solde initial de 0
    4. Le systeme retourne le compte cree avec son ID unique
- **Scenario alternatif** : Donnees manquantes -> erreur 400

CU02 - Lister les comptes

- **Acteur** : Client
- **Pre-condition** : Aucune
- **Scenario principal** :
    1. Le client envoie une requete GET /api/comptes
    2. Le systeme retourne la liste de tous les comptes

CU03 - Consulter un compte

- **Acteur** : Client
- **Pre-condition** : Le compte existe
- **Scenario principal** :
    1. Le client envoie une requete GET /api/comptes/:id
    2. Le systeme retourne les details du compte
- **Scenario alternatif** : Compte introuvable -> erreur 404

CU04 - Effectuer un depot

- **Acteur** : Client
- **Pre-condition** : Le compte existe, montant > 0
- **Scenario principal** :
    1. Le client envoie POST /api/comptes/:id/depot avec le montant
    2. Le systeme valide le montant
    3. Le systeme ajoute le montant au solde
    4. Le systeme enregistre la transaction et retourne le compte mis a jour
- **Scenario alternatif** : Montant invalide -> erreur 400

CU05 - Effectuer un retrait

- **Acteur** : Client
- **Pre-condition** : Le compte existe, montant > 0, solde suffisant
- **Scenario principal** :
    1. Le client envoie POST /api/comptes/:id/retrait avec le montant
    2. Le systeme verifie que le solde est suffisant
    3. Le systeme soustrait le montant du solde
    4. Le systeme enregistre la transaction et retourne le compte mis a jour
- **Scenarios alternatifs** :
    - Montant invalide -> erreur 400
    - Solde insuffisant -> erreur 400

CU06 - Consulter l'historique des transactions

- **Acteur** : Client
- **Pre-condition** : Le compte existe
- **Scenario principal** :
    1. Le client envoie GET /api/comptes/:id/transactions
    2. Le systeme retourne la liste des transactions du compte

---

5. Endpoints de l'API

Methode	Endpoint	Description
POST	/api/comptes	Creer un compte
GET	/api/comptes	Lister tous les comptes
GET	/api/comptes/:id	Consulter un compte

Methode	Endpoint	Description
POST	/api/comptes/:id/depot	Effectuer un depot
POST	/api/comptes/:id/retrait	Effectuer un retrait
GET	/api/comptes/:id/transactions	Historique des transactions

---

6. Modeles de donnees

Compte

```
{
  "id": "uuid",
  "nom": "string",
  "prenom": "string",
  "solde": "number",
  "dateCreation": "ISO 8601"
}
```

Transaction

```
{
  "id": "uuid",
  "compteId": "uuid",
  "type": "depot | retrait",
  "montant": "number",
  "date": "ISO 8601"
}
```

---

7. Stack technique

- **Runtime** : Node.js
- **Framework** : Express.js
- **Stockage** : En memoire (evolutif vers une BDD)
- **Format** : JSON
- **Documentation** : Swagger (OpenAPI 3.0)

---

8. Plan de tests (ICT304 - Software Testing)

Les cas de tests suivent le **Test Case Format (Template 1)** du cours ICT304 et appliquent le principe du **Defensive Testing** : couverture des conditions normales (entrees valides) et anormales (entrees invalides).

8.1 Legende

- **Test Case ID** : Identifiant unique
- **Test Case Description** : Objectif du test
- **Test Input** : Donnees d'entree
- **Expected Results** : Resultat attendu (code HTTP + message)
- **Test Case Status** : Not Executed / Pass / Fail

8.2 Module General

Test Case ID	Test Case Description	Test Input	Expected Results	Status
TC-001	Verifier la page d'accueil	`GET /`	HTTP 200 + message de bienvenue	Not Executed
TC-002	Acceder a la documentation Swagger	`GET /api-docs`	HTTP 200 + Swagger UI	Not Executed

8.3 Module Creation de compte (POST /api/comptes)

Test Case ID	Test Case Description	Test Input	Expected Results	Status
TC-003	Creer un compte valide	`{nom: "Dupont", prenom: "Jean"}`	HTTP 201 + compte cree avec UUID, solde 0.00 FCFA	Not Executed
TC-004	Creer sans champ nom	`{prenom: "Jean"}`	HTTP 400 + erreur champs requis	Not Executed
TC-005	Creer sans champ prenom	`{nom: "Dupont"}`	HTTP 400 + erreur champs requis	Not Executed
TC-006	Creer avec corps vide	`{}`	HTTP 400 + erreur	Not Executed
TC-007	Creer avec nom vide	`{nom: "", prenom: "Jean"}`	HTTP 400 + erreur	Not Executed
TC-008	Creer avec prenom vide	`{nom: "Dupont", prenom: ""}`	HTTP 400 + erreur	Not Executed

Test Case ID	Test Case Description	Test Input	Expected Results	Status
TC-009	Creer deux comptes (UUIDs uniques)	Deux requetes valides	Deux UUIDs differents	Not Executed
TC-010	Creer sans Content-Type JSON	Sans header JSON	HTTP 400	Not Executed

8.4 Module Listing (GET /api/comptes)

Test Case ID	Test Case Description	Test Input	Expected Results	Status
TC-011	Lister quand aucun compte	`GET /api/comptes`	HTTP 200 + tableau vide	Not Executed
TC-012	Lister 3 comptes	Apres 3 creations	HTTP 200 + 3 comptes	Not Executed
TC-013	Verifier formatage du solde	`GET /api/comptes`	Chaque solde au format "X.XX FCFA"	Not Executed

8.5 Module Consultation (GET /api/comptes/:id)

Test Case ID	Test Case Description	Test Input	Expected Results	Status
TC-014	Consulter compte existant	ID UUID valide	HTTP 200 + donnees compte	Not Executed
TC-015	Consulter ID inexistant	UUID non cree	HTTP 404 + "Compte introuvable"	Not Executed
TC-016	Consulter ID malforme	`GET /api/comptes/abc`	HTTP 404	Not Executed

8.6 Module Depot (POST /api/comptes/:id/depot)

Test Case ID	Test Case Description	Test Input	Expected Results	Status
TC-018	Depot valide	`{montant: 5000}`	HTTP 200 + solde mis a jour	Not Executed
TC-019	Depot sur compte inexistant	ID inconnu	HTTP 404	Not Executed
TC-020	Depot avec montant negatif	`{montant: -100}`	HTTP 400 + "montant doit etre positif"	Not Executed

Test Case ID	Test Case Description	Test Input	Expected Results	Status
TC-021	Depot avec montant zero	`{montant: 0}`	HTTP 400	Not Executed
TC-022	Depot avec chaine	`{montant: "5000"}`	HTTP 400 (typeof != number)	Not Executed
TC-023	Depot sans montant	`{}`	HTTP 400	Not Executed
TC-024	Depot avec null	`{montant: null}`	HTTP 400	Not Executed
TC-025	Depots successifs (cumul)	1000 puis 2000	Solde = 3000.00 FCFA	Not Executed
TC-026	Depot tres grand	`{montant: 1000000000}`	HTTP 200	Not Executed
TC-027	Depot decimal	`{montant: 1234.56}`	HTTP 200 + "1234.56 FCFA"	Not Executed

8.7 Module Retrait (POST /api/comptes/:id/retrait)

Test Case ID	Test Case Description	Test Input	Expected Results	Status
TC-028	Retrait valide	solde=10000, `{montant: 3000}`	HTTP 200 + solde = 7000.00 FCFA	Not Executed
TC-029	Retrait compte inexistant	ID inconnu	HTTP 404	Not Executed
TC-030	Solde insuffisant	solde=1000, `{montant: 5000}`	HTTP 400 + "Solde insuffisant"	Not Executed
TC-031	Retrait = solde (limite)	solde=5000, `{montant: 5000}`	HTTP 200 + solde = 0.00 FCFA	Not Executed
TC-032	Montant negatif	`{montant: -500}`	HTTP 400	Not Executed
TC-033	Montant zero	`{montant: 0}`	HTTP 400	Not Executed
TC-034	Sans montant	`{}`	HTTP 400	Not Executed
TC-035	Montant non numerique	`{montant: "abc"}`	HTTP 400	Not Executed
TC-036	Retraits successifs	solde=1000, 2x500	Solde final = 0	Not Executed

Test Case ID	Test Case Description	Test Input	Expected Results	Status
TC-037	Retrait sur solde=0	`{montant: 100}`	HTTP 400 + "Solde insuffisant"	Not Executed

8.8 Module Historique (GET /api/comptes/:id/transactions)

Test Case ID	Test Case Description	Test Input	Expected Results	Status
TC-038	Historique vide	Compte neuf	HTTP 200 + tableau vide	Not Executed
TC-039	Historique apres 1 depot	1 depot de 5000	HTTP 200 + 1 transaction "depot"	Not Executed
TC-040	Historique mixte	1 depot + 1 retrait	HTTP 200 + 2 transactions	Not Executed
TC-041	Historique compte inexistant	ID inconnu	HTTP 404	Not Executed
TC-042	Isolation entre comptes	Compte A avec txns, B vide	B reste vide	Not Executed
TC-043	Formatage montant/date	Transaction existante	Montant "X.XX FCFA" + date FR	Not Executed

8.9 Scenarios bout-en-bout (Integration)

Test Case ID	Test Case Description	Test Input	Expected Results	Status
TC-044	Scenario complet	Creation + depot + retrait + historique	Toute la chaine coherente	Not Executed
TC-045	Creations simultanees	5 POST paralleles	5 UUIDs uniques	Not Executed
TC-046	Transactions concurrentes	2 depots paralleles 1000	Solde = 2000.00 FCFA	Not Executed

8.10 Matrice de tracabilite (RTM)

Conformement au cours ICT304, chaque exigence fonctionnelle est liee a ses cas de tests :

Besoin	Description	Cas de tests
BF01	Creer un compte	TC-003 a TC-010
BF02	Consulter la liste des comptes	TC-011 a TC-013

Besoin	Description	Cas de tests
BF03	Consulter un compte par ID	TC-014 a TC-016
BF04	Effectuer un depot	TC-018 a TC-027
BF05	Effectuer un retrait	TC-028 a TC-037
BF06	Consulter l'historique	TC-038 a TC-043
BNF02	Validation des donnees	TC-004 a TC-008, TC-020 a TC-024, TC-032 a TC-035
BNF03	Codes HTTP appropries	Tous les TC
BNF04	Solde jamais negatif	TC-030, TC-031, TC-037
BNF05	Documentation API	TC-001, TC-002

8.11 Synthese

- **Nombre total de cas de tests** : 46
- **Couverture** : defensive (normale + anormale), limites, integration
- **Types d'erreurs testees** : syntaxiques, logiques, semantiques, limites